Context-Scoped AI Agents in Integrated Developer Workspaces: A Between-Subjects Study of Information Breadth and Task Performance

ANDREW SADLER, Michigan Technological University, USA

JIE JW WU, Michigan Technological University, USA

Software development is fragmented across issue trackers, team chat, and code hosts, and AI assistants embedded in that workflow face an analogous fragmentation: they can only act on what they are allowed to see. We operationalise this as a *context ladder*—a three-rung experimental manipulation of how much of a shared workspace an AI agent can observe and act on: chat only (C_1), chat and task board (C_2), or chat, board, and GitHub (C_3)—and measure the downstream effect on user task performance, cognitive load, reliance, and trust across a population spanning both novice and experienced programmers.

We present Weld, an open-source integrated developer workspace (SvelteKit + Rust/Axum) combining a Kanban board, team chat, and mocked GitHub repository data in one product, and report a hybrid user study (target N = 20–30) recruiting from two complementary populations: university CS students (undergraduate and graduate, representing less-experienced developers) and professional software developers ($\geq$1 year of industry experience). Every participant first completes six manual no-agent baseline tasks— establishing a per-person speed and workload anchor that absorbs individual skill differences—then is randomly assigned to one agent condition for six agent-assisted tasks spanning comprehension, single-surface, and cross-surface coordination types. Including both populations allows us to examine whether the benefit of context breadth varies with developer experience, and whether novice and experienced programmers differ in their reliance on and trust of an AI agent with varying information access. Primary analysis uses baseline-normalised task time via a mixed-effects model testing the condition × task-type interaction; secondary measures include NASA-TLX, System Usability Scale, a five-item integration-perception scale, and an eight-item AI trust scale.

Context scoping is enforced at the server in code—not in prompts—by the Context Gate module, which records a per-turn `agent_context_sent` audit snapshot proving, from the log alone, exactly what information each agent invocation received. This architectural choice makes the experimental manipulation verifiable rather than merely assumed. We contribute the context-ladder study design, the mixed-experience participant protocol, and the Context Gate mechanism as replicable instruments for empirical research on AI agents in multi-surface collaborative tools.

CCS Concepts: • **Software and its engineering** → **Software development techniques**; • **Human-centered computing** → **HCI theory, concepts and models**; **Empirical studies in HCI**; • **Computing methodologies** → *Artificial intelligence.*

Additional Key Words and Phrases: AI agents, developer tools, integrated workspaces, context scoping, user study, NASA TLX, cognitive load, task coordination, human-AI interaction, software engineering

ACM Reference Format:

Andrew Sadler and Jie JW Wu. 2026. Context-Scoped AI Agents in Integrated Developer Workspaces: A Between-Subjects Study of Information Breadth and Task Performance. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (June 2026), 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Authors' Contact Information: Andrew Sadler, awsadler@mtu.edu, Michigan Technological University, Houghton, Michigan, USA; Jie JW Wu, jie.jw.wu@gmail.com, Michigan Technological University, Houghton, Michigan, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

Modern software development demands constant movement across tools: a task management board to plan work, a chat platform to coordinate with teammates, and a code-hosting service to review pull requests and track issues. Each transition carries a context-switching cost—the cognitive overhead of re-establishing mental state in a new tool—that fragments attention and increases workload [4, 12]. This fragmentation is compounded when information lives in separate silos: acting on a chat message that references a board card requires the developer to open two applications and mentally merge their contents. Ko et al. [8] documented this cross-tool information-seeking overhead in collocated software teams; LaToza et al. [9] found it a primary driver of knowledge loss and coordination breakdowns.

Integrated developer workspaces—products that co-locate boards, chat, and code review—reduce this overhead by design, and they also create a new possibility: an embedded AI agent that can *observe and act* across all surfaces at once. How much of that workspace such an agent should be allowed to see, and what that breadth buys the user, remains thin in the empirical record. Prior AI-productivity research has focused on single-surface tasks (code generation, text drafting), leaving the cross-surface coordination layer largely unstudied [3, 13]. Agentic frameworks such as ReAct [15] and Reflexion [14] show strong performance on synthetic benchmarks but have not been evaluated with human participants on realistic multi-surface workflows.

A second open question concerns *context breadth*: how much workspace state should an agent be allowed to observe? Wider context may improve agent relevance but raises concerns about auditability and trust. Existing systems gate context via prompt instructions, which are opaque and unverifiable—there is no way to audit, after the fact, exactly what information an agent received during a specific turn.

This paper makes three contributions:

(1) **Weld system.** We describe Weld, an open-source all-in-one developer workspace (Kanban board, team chat, GitHub integration) with an embedded study mode that provisions isolated per-participant workspaces, implements balanced condition assignment, and logs fine-grained behavioral telemetry (§3).
(2) **Code-enforced context scoping (Context Gate).** We introduce the Context Gate, a server-side module that assembles agent context from a condition-specific database allowlist, with a per-turn audit snapshot (`agent_context_sent`) that proves—from the server log alone— exactly which information sources a given agent turn could access (§3).
(3) **Empirical study.** We report a hybrid within/between-subjects user study (N = *TBD*) that, against each participant's own no-agent baseline, crosses three agent-context conditions (C_1 chat-only advisory; C_2 chat+board action; C_3 chat+board+GitHub full) with three task types (comprehension, single-surface, cross-surface) to ask whether broader context makes users more effective (RQ1), how it shifts their reliance on versus verification of the agent (RQ2), and whether their trust tracks the agent's measured effectiveness (RQ3) (§4, §5).

The manipulated variable is the agent's context breadth alone. We make no claim that an all-in-one workspace outperforms separate tools—this design has no fragmented-tooling arm to test that—and treat consolidation as the apparatus and motivation, not as a hypothesis.

The remainder of the paper is structured as follows. Section 2 surveys related work on tool fragmentation, AI productivity agents, and context management. Section 3 describes Weld and its study infrastructure. Section 4 presents the study design. Section 5 reports results. Section 6 discusses implications, and Section 7 concludes.

2 Related Work

2.1 Fragmentation and Context-Switching in Software Development

The cognitive cost of task interruption and tool-switching in knowledge work has been well-documented. Mark et al. [12] showed through in-situ observation that interrupted workers worked faster to compensate but reported higher stress and effort. Czerwinski et al. [4] found through diary studies that workers switched tasks on average every three minutes, with complex task resumption taking substantially longer than simple ones. In software engineering specifically, Ko et al. [8] identified information fragmentation across tools as a primary bottleneck for collocated teams: developers spent significant time locating and reconciling information scattered across email, wikis, issue trackers, and chat. LaToza et al. [9] found that maintaining accurate mental models across multiple tools was a persistent challenge, with developers frequently reverting to conversations to reconstruct shared state. Together, these findings motivate the core premise of integrated developer workspaces: that reducing inter-tool navigation distance reduces cognitive load and improves task throughput.

Commercial responses—Linear (boards + notifications), Notion (docs + tasks), GitHub Projects (issues + boards), Slack with app integrations—have proliferated. Whether such consolidation outperforms separate tools is a question this study does *not* attempt to answer: it has no fragmented-tooling arm. Instead, the integrated workspace is our apparatus, and the no-agent baseline phase serves as a per-person speed and workload anchor for the agent-context manipulation that follows—not as a controlled integration-versus-fragmentation comparison.

2.2 AI Agents for Developer Productivity

The introduction of large language model (LLM) assistants into developer workflows has produced a growing body of empirical evidence. Chen et al. [3] demonstrated that LLMs could generate functionally correct code for a substantial fraction of benchmark problems, and GitHub Copilot [6] subsequently reported that developers accepted a significant fraction of AI-generated suggestions in production use. Noy and Zhang [13] showed in a randomized controlled trial that access to an LLM writing assistant improved task completion time and output quality for college-educated workers. These studies focus on single-surface, largely *generative* tasks—writing code, text, or structured content.

The evaluation of AI agents in *coordination* tasks—where the agent must synthesize information across surfaces and take multi-step actions—is less developed. Work on agentic LLM frameworks such as ReAct [15] and Reflexion [14] demonstrates strong performance on synthetic benchmark tasks (web navigation, knowledge-base QA, code execution), but these benchmarks do not capture the human-in-the-loop, multi-surface collaboration dynamics present in real developer work. Studies of LLM agents in CSCW contexts—where an agent participates in a team workflow rather than assisting a single user on a single surface—are largely absent from the literature.

Weld's embedded agent operates in this understudied space: it can read the team's chat history, inspect and modify the Kanban board, and link cards to GitHub artifacts—all within a single turn. Crucially, the agent's actions are not auto-applied; board mutations and drafted chat messages are surfaced to the participant for review, keeping the human in the loop for consequential actions. This design reflects recent findings [13] that humans are effective editors of AI-generated outputs even when they cannot independently generate them.

2.3 Context Management in LLM-Based Systems

A central design challenge for LLM-powered tools is *context management*: deciding what information to include in the model's context window and at what granularity. In retrieval-augmented generation (RAG) systems [10], this is

handled by a retrieval step that selects relevant passages from a corpus. In multi-agent systems, each agent typically receives a curated subset of shared state. The trade-off between context richness and noise (or cost, in token-priced systems) has been analysed from an accuracy standpoint but rarely from the perspective of user trust, cognitive load, or task-type interaction.

Prior deployed systems typically gate context in one of two ways. The first approach is *prompt-level* gating: the system prompt instructs the model to ignore or not request certain information sources. This is easy to implement but is neither verifiable nor robust—the model may still reference disallowed information if it was included upstream, and there is no audit trail proving that a given agent turn operated within intended bounds. The second approach is *integration-level* gating: only the API calls corresponding to allowed sources are made before the model is invoked. This is the approach taken by Weld's Context Gate.

The auditability of context decisions is closely related to the broader theme of transparency in AI systems. Dey's foundational framework [5] defines context as "any information that can be used to characterise the situation of an entity," and argues that context-aware systems must make their information selection criteria explicit to maintain user trust. Cavoukian's privacy-by-design principles [2] extend this to governance: systems should embed privacy and data minimisation into their architecture, not bolt it on. The Context Gate is an instance of this principle applied to agent information access: what the agent cannot see by construction, it cannot leak, misuse, or be manipulated into retrieving.

2.4 User Studies of AI-Augmented Tools

Studies of AI-augmented developer tools have primarily measured productivity via proxy (task completion rate, time, code quality). Fewer studies employ validated psychometric instruments alongside behavioral measures. Our study combines NASA-TLX [7]—a widely used multidimensional workload scale—with the System Usability Scale [1] and a trust scale adapted from Madsen and Gregor [11], enabling comparison with prior HCI work on tool usability and AI trust. The pairing of subjective self-report scales with objective behavioral telemetry (time-on-task, cross-surface event counts) allows triangulation that neither source alone affords: participants may report low workload on tasks where telemetry shows long completion times, or vice versa, and this discrepancy is itself informative.

3 System: Weld

3.1 Architecture Overview

Weld is an open-source, full-stack developer workspace targeting teams of two to ten engineers. The web frontend is built with SvelteKit 2 (TypeScript, Tailwind CSS 4); the backend is an Axum 0.8 server (Rust) backed by PostgreSQL 16 and Redis. A WebSocket hub broadcasts real-time board and chat updates to all connected clients of a team. All study-mode paths share the same production application binary, activated by an `appConfig.studyMode` flag.

Weld exposes three collaboration surfaces that correspond exactly to the three context sources manipulated across the agent conditions:

Kanban Board. Columns and cards with fractional-indexed positions, real-time drag-and-drop reordering, label and priority assignment, and GitHub issue/PR links. Cards carry a `github_issue` and `github_pr` field that are populated either by the user or by the agent's `link_card_to_github` tool. Cross-references to chat messages render inline as `[[card:uuid]]` tokens that resolve to card titles on hover.

Team Chat. Channel-based messaging with threading, Markdown rendering, @mention resolution, and the same `[[card:uuid]]` cross-reference syntax used on the board. Messages are delivered over WebSocket and persisted

server-side; the agent's `post_message` tool drafts a message that the participant must review and confirm before it is sent.

GitHub Integration. A GitHub App installation synchronizes open issues and pull requests into Weld's read-only GitHub view. Two-way linking (card ↔ issue, card ↔ PR) is supported both manually and via the agent's `link_card_to_github` tool.

3.2 Study Mode and Workspace Isolation

Each study participant is provisioned in a single atomic database transaction via the `provision_participant` function. The transaction creates: a dedicated user account, a team with four pre-seeded synthetic teammates (Sarah Chen, Alex Rivera, Priya Nair, and Marcus Webb), a PetTracker project, three board columns (Backlog, In Progress, Done), 15–20 cards with realistic descriptions and priorities, two channels (#general and #pettracker) with pre-seeded message history, and GitHub fixture data. All timestamps in the seeded content are stored as negative offsets from the provisioning moment, so the workspace always appears to have been active for the same relative period regardless of when the participant joins.

Access codes and resumption. Participants authenticate via single-use access codes rather than email/password registration. If a participant closes the browser mid-session, they can return to the same code on any device and resume from their saved stage and task index (`resume_stage`, `resume_task_index`). Progress is persisted by `update_progress(pool, user_id, stage, task_index)`.

Condition assignment. The `pick_agent_condition` function implements balanced permuted-block randomization with block size 3 and a random tie-break on equal counts, ensuring that no condition falls behind the others as enrollment proceeds while avoiding deterministic C1→C2→C3 cycling that could confound condition with enrollment order.

Database isolation. Every query in study mode is scoped by `team_id` and `project_id` in the SQL `WHERE` clause. No cross-participant query paths exist; the study database schema enforces this at the row level.

3.3 The Context Gate

The Context Gate is a Rust module (`services/study/agent/gate.rs`) that acts as the sole chokepoint for assembling the agent's context before each LLM call. Its design centers on two structures: `ContextRules`, which encodes the per-condition allowlist, and `ScopedContext`, the assembled result.

Rules and allowlists. `rules_for(condition)` returns a `ContextRules` value specifying which of the three sources (chat, board, GitHub) may be queried. The rules for each condition are:

Condition	Chat	Board	GitHub
C_1 (advisory)	✓	–	–
C_2 (action)	✓	✓	–
C_3 (full)	✓	✓	✓

The gate's `assemble(pool, ctx)` function queries *only* the allowed sources; the database calls for disallowed sources are never made. The resulting `ScopedContext` carries a `sources_included` field listing exactly which sources were consulted, a `chat: Vec<ChatLine>` field, an `board: Option<Vec<CardLine»`, and a `github: Option<Vec<GithubItem»`. Disallowed fields are `None`.

Proof-of-scoping snapshot. Before invoking the LLM, `run_turn` calls `ScopedContext::snapshot()`, which serialises the assembled context (minus any fields that should not appear in logs) and records it as an `agent_context_sent` telemetry event. This creates an immutable per-turn audit record proving that the context assembly respected the assigned condition.

Double-gated tool executor. The gate also governs which tools are available to the model. `allowed_tool_categories(condition)` returns a static slice of `ToolCategory` variants: C_1 receives `[]`, C_2 receives `[BoardWrite, ChatPost]`, and C_3 receives all three categories (`[BoardWrite, ChatPost, GithubLink]`). This list determines which tool definitions are sent to the model in the OpenAI request (first gate). The tool executor (`tools::execute`) independently re-validates the tool name against the condition's allowed categories before executing any action; any call that slips past the first gate is rejected and logged as `agent_tool_rejected` (second gate). Table 1 lists the five tools and their condition availability.

Table 1. Agent tools by condition availability. ✓ = available, – = not offered to model and rejected at executor.

Tool name	Action	C_1	C_2	C_3
`create_card`	Create a board card	–	✓	✓
`move_card`	Move a card to another column	–	✓	✓
`update_card`	Edit card priority / assignee / description	–	✓	✓
`post_message`	Draft a chat message for review	–	✓	✓
`link_card_to_github`	Link a card to an issue or PR	–	–	✓

Turn budget. Each agent invocation is bounded by two hard limits: `MAX_TURNS = 5` model↔tool round-trips per invocation, and `MAX_AGENT_MESSAGES = 50` total agent messages per participant across the entire session. These limits prevent runaway loops and bound infrastructure cost.



Fig. 1. The Context Gate architecture. The `assemble()` call queries only allowed sources for the participant's assigned condition and emits a `agent_context_sent` snapshot. The double-gated tool executor re-validates tool permissions independently at execution time, logging violations as `agent_tool_rejected` events.

3.4 Agent Turn Orchestration

The `run_turn` function orchestrates each agent invocation. After assembling the `ScopedContext` and recording the `agent_context_sent` event, it loads prior turns from the `study_agent_turns` table to maintain conversation continuity across the session. The function then enters a loop bounded by `MAX_TURNS`, alternating between LLM calls (with condition-appropriate tool definitions from `openai_tool_defs(condition)`) and tool execution calls.

Condition-specific system prompt. The system prompt varies by condition. For C_1, it instructs the model to "provide guidance but not suggest or execute actions," constraining it to an advisory role even though no tools are offered. For C_2 and C_3, the prompt frames the agent as "a co-worker in the team workspace" that can take actions on behalf of the participant. This framing difference is intentional: it matches participant expectations to the agent's actual capability.

Chat message drafting. The `post_message` tool does not immediately post a message; instead, the tool executor returns a `ChatDraft` to the client, which renders it as a review panel. The participant must explicitly confirm before the message is sent. This keeps human oversight over public communication while still reducing the mechanical effort of composing and locating the correct channel.

3.5 Telemetry Infrastructure

Weld records 43 discrete event types (Table 2). Client events are batched and submitted to a single endpoint that validates each event type against a server-side allowlist (`ALLOWED_EVENT_TYPES`) before persistence; events not on the allowlist are silently dropped. This *allowlist-by-construction* approach means that even a modified client cannot introduce new event types, preventing accidental capture of keystrokes, scroll positions, or other data outside the study protocol.

Server-side events (agent lifecycle, context snapshots, tool outcomes) are recorded via `record_server_event` directly within the Axum handlers, bypassing the client batch path and therefore unaffected by client connectivity or manipulation.

4 Study Design

4.1 Research Questions

The study manipulates exactly one variable—how much of the shared workspace the embedded agent can observe and act on (the $C_1 \rightarrow C_2 \rightarrow C_3$ context ladder)—measured against each participant's own no-agent baseline (C0). The questions stay inside what that manipulation supports: how much context an embedded agent needs and how users work with it. We deliberately make *no* claim that an all-in-one workspace outperforms separate tools; this design has no fragmented-tooling arm to test that, and consolidation serves only as the apparatus and motivation (§2.1), not as a hypothesis.

RQ1 (primary) — Context breadth → effectiveness. Does broadening the agent's context (chat → +board → +GitHub) make the user more effective, and on which task types?

RQ2 — Reliance. How much do users delegate to versus verify and override the agent, and does that division of labor shift with the agent's context breadth?

RQ3 — Trust versus reality. Does users' trust in the agent track its measured effectiveness, or diverge from it?

Table 2. Telemetry event categories and types recorded by Weld.

Category	Event types
Session & phase	`session_start`, `session_end`, `session_abandoned`, `session_resumed`, `phase_started`, `phase_ended`, `consent_given`
Survey	`survey_started`, `survey_completed`
Task lifecycle	`task_shown`, `task_completed`, `task_skipped`, `task_final_state`, `task_rating`, `instruction_reopened`
Board	`card_created`, `card_opened`, `card_edited`, `card_moved`, `card_assigned`, `card_labeled`, `card_priority_set`, `card_deleted`, `card_searched`, `board_filtered`
Chat	`message_sent`, `message_edited`, `message_deleted`, `channel_switched`, `thread_opened`, `mention_clicked`, `link_clicked`
Cross-surface	`card_referenced_from_chat`, `chat_referenced_from_card`, `pr_referenced_from_card`, `card_referenced_from_pr`, `issue_referenced_from_card`
GitHub	`github_view_opened`, `pr_opened`, `issue_opened`, `pr_linked_to_card`, `issue_linked_to_card`
Navigation	`view_changed`, `search_used`
Agent	`agent_invoked`, `agent_context_sent`, `agent_response`, `agent_tool_call`, `agent_tool_rejected`, `agent_suggestion_accepted`, `agent_suggestion_rejected`, `agent_action_undone`, `agent_output_edited`, `agent_error`, `agent_panel_opened`, `agent_panel_collapsed`

4.2 Hypotheses

H1 (RQ1, speed). Broader agent context yields a larger improvement over the participant's own C0 baseline (faster completion): $C_3 > C_2 > C_1$ in baseline-normalized speed.

H2 (RQ1, task-type interaction). The gain is *task-dependent*: negligible on comprehension and single-surface tasks (where cross-surface data is irrelevant) but substantial on cross-surface tasks that span chat, board, and GitHub. This condition × task-type interaction is the key result.

H3 (RQ1, cognitive load). Broader context lowers self-reported cognitive load (NASA-TLX) relative to baseline more than narrow context does, driven primarily by the effort and mental-demand subscales.

H4 (RQ2, reliance calibration). Delegation to the agent rises with context breadth, but reliance is imperfectly calibrated: users do not verify or override precisely where the agent is actually wrong.

H5 (RQ3, perception–behavior gap). Trust and perceived speedup need not track the agent's measured effectiveness: participants may trust an agent that *does more* even when it is no faster, or is occasionally wrong. This perception–behavior divergence is the study's most novel angle.

4.3 Design Overview

The study is a **hybrid within/between-subjects** design. Each participant first completes six manual baseline tasks with no agent (the within-subjects C0 anchor), then is randomly assigned to one of three agent-context conditions and completes six agent-assisted tasks in that fixed condition (the between-subjects manipulation). The per-person C0 baseline lets each agent-task time be analyzed as a *delta from that participant's own manual speed*, absorbing the individual skill variance that would otherwise sit between the between-subjects cells and buying back much of the statistical power a pure between-subjects design loses. Holding the agent's access fixed per person avoids within-person

contamination—once a participant has seen the full-context agent solve a task, they cannot un-see how it should go—and keeps the session within a self-serve ~55–65-minute budget.

4.4 Participants

We target N = 20–30 participants drawn from two complementary populations: *university CS students* (undergraduate and graduate, recruited via course mailing lists) and *professional software developers* (≥1 year of industry experience, recruited via professional networks and online communities). Recruiting both groups lets us examine whether the benefit of context breadth—and reliance on and trust of the agent—varies with prior experience. Eligibility further required self-reported familiarity with at least one task-management tool (e.g., Jira, Linear, GitHub Projects) and one team-chat tool (e.g., Slack, Teams, Discord). The study was conducted remotely. Participants were compensated *[amount TBD]*.

4.5 Conditions

Participants were randomly assigned, with balanced permuted-block randomization (block size 3), to one of three between-subjects conditions:

- C_1 **(Chat only, advisory).** The agent observes only team chat history. No access to board state or GitHub. Advisory-only: no tools are offered to the model, so it can advise but cannot modify any surface.
- C_2 **(Chat + Board, action-capable).** The agent observes chat and the full board card state. Available tools: `create_card`, `move_card`, `update_card`, and `post_message` (drafts a chat message for the participant to review before sending).
- C_3 **(Chat + Board + GitHub, full action).** The agent observes all three surfaces. All C_2 tools plus `link_card_to_github`, enabling it to connect board cards to GitHub issues or pull requests.

Capability scales with context—by design. An agent cannot act on a board it cannot see, so visibility and action capability are intentionally coupled along the ladder. This bundling *is* the phenomenon under study ("does giving the agent more access help the user"); we state it as a design decision, not a confound to be removed.

4.6 Tasks

Each participant completes **12 tasks in fixed order**: six manual *baseline tasks* (B1–B6, C0, no agent) followed by six *agent-assisted tasks* (T1–T6) in their assigned condition. Each baseline is built to match an agent task type, giving a like-for-like per-person normalizer, while baseline and agent content are *disjoint* to prevent learning transfer. The agent phase uses the `gpt-4o-mini-2024-07-18` model at temperature 0.2 with versioned, condition-specific system prompts pinned for reproducibility.

Tasks span three types across both phases:

Comprehension. Synthesize information from one or more surfaces to form a judgment (e.g., "what does the team need most right now?"). All conditions should do well; chat-only is competitive.

Single-surface. Completable by acting on a single surface; extra context should add little. These serve as controls showing broader context does *not* universally help.

Cross-surface. Explicitly requires coordinated action across two or more surfaces. Where H1/H2 live: full context should pull ahead, chat-only should struggle.

Table 3. Task design summary. B = Baseline (no agent, C0), T = Agent task. Surfaces: Ch = Chat, Bo = Board, GH = GitHub. T6 is constructed so the correct PR is identifiable *only* with GitHub context—the single cleanest test of the top of the ladder.

ID	Phase	Type	Surfaces	Summary
B1	Baseline	Single-surface	Ch, Bo	Find blocked card; move to In Progress
B2	Baseline	Cross-surface	Ch, Bo	Check a card's column; post status reply
B3	Baseline	Cross-surface	Ch, Bo	Open card from chat link; set priority
B4	Baseline	Cross-surface	GH, Ch	Find open PR in GitHub view; post number in chat
B5	Baseline	Cross-surface	Ch, Bo	Recover a decision from chat; record on a card
B6	Baseline	Comprehension	Bo, Ch	Find CI-pipeline card; confirm what it does in chat
T1	Agent	Comprehension	Ch, Bo	"What needs doing?" then act
T2	Agent	Comprehension	Ch, Bo, GH	"Catch me up": summarize recent activity
T3	Agent	Single-surface	Ch	Draft and post a status update in chat
T4	Agent	Single-surface	Bo	Set priorities on backlog cards
T5	Agent	Cross-surface	Ch, Bo	Take a bug from chat to the board
T6	Agent	Cross-surface	Ch, Bo, GH	Link the PR to the card; note it in chat

Because condition is between-subjects, all participants see the *same* tasks; only their agent's access differs. Task content is thus held constant and context breadth is the sole manipulation.

4.7 Procedure

Each session proceeded as follows:

(1) Informed consent and eligibility screening.
(2) Orientation to Weld (un-timed, guided).
(3) **Baseline phase** (6 tasks, no agent, condition C0): tasks presented sequentially; participants self-paced and rated each task before advancing.
(4) **Post-baseline interstitial**: a NASA-TLX instrument is administered here, *before any agent exposure* (`tlx_*_baseline`), capturing manual-phase load. The agent panel is then revealed; C_2/C_3 participants receive a brief explanation of the available agent actions.
(5) **Agent phase** (6 tasks): same structure, with the agent available under the participant's assigned condition.
(6) Exit survey: a second NASA-TLX (`tlx_*`, agent phase), SUS, integration-perception items, the AI-trust block, open-ended comments, and demographics.
(7) Debrief.

Sessions were conducted remotely. Screen recording was *not* collected; all behavioral data came from in-product telemetry (§3.5).

4.8 Measures

Task performance (primary, RQ1). End-to-end *active* time-on-task per task—the client clock runs only while the tab is visible and persists across reloads, so idle and away time are excluded by construction (`task_completed.task_time_ms`). The headline outcome is **baseline-normalized time**: each agent-task time divided by that participant's own type-matched C0 baseline—comprehension (T1, T2) ÷ B6, single-surface (T3, T4) ÷ B1, cross-surface (T5, T6) ÷ mean(B2–B5). Completion is judged from `task_final_state`.

Correctness (RQ1, quality control). Per-task correctness on a 0–3 rubric, hand-coded by two raters (Cohen's κ on a shared subset). Always reported beside time: a fast-but-wrong agent is not effective.

Cognitive load (H3). NASA-TLX [7], six subscales, collected **twice**— post-baseline (`tlx_*_baseline`) and at exit (`tlx_*`)—so load is analyzed as a within-person agent−baseline delta rather than a single end-of-session snapshot. A per-task effort rating (1–7, every task including baselines) provides a finer-grained, task-localized companion measure.

Reliance and division of labor (RQ2). From telemetry: the *delegation ratio* (agent- vs. user-performed actions), verification behavior (re-reads, edits to the agent's output, undo), and acceptance vs. override of agent suggestions (`agent_suggestion_accepted/_rejected`, `agent_output_edited`, `agent_action_undone`), plus view-switching and cross-reference usage.

Trust (RQ3). An eight-item AI-trust block (adapted from [11]) covering reliability, predictability, competence, willingness to rely without checking, and whether the agent had enough context, plus `trust_speed_perceived`. The gap between perceived speedup/trust and measured normalized time/correctness operationalizes H5.

Usability and integration perception (descriptive). SUS [1] (10 items) and a five-item integration-perception scale; `int_switch_perceived` is paired with the behavioral view-switch rate. These describe the apparatus and are not RQ outcomes.

Scoping manipulation check. The per-turn `agent_context_sent` snapshot (§3.3) provides direct evidence that the IV held: every C_1 snapshot must contain no board or GitHub context, and every C_2 snapshot no GitHub context. Reported before any inferential result.

4.9 Analysis

The primary RQ1 model is a linear mixed model on log-transformed normalized time: `log(norm_time) ~ condition * task_type + (1 | participant)`, with the condition × task-type interaction as the H2 test. Correctness is modeled in parallel with an ordinal cumulative-link mixed model. H3 compares the within-person NASA-TLX delta between conditions. RQ2 is reported as reliance descriptives (delegation ratio, the invoke → execute → accept/edit/undo funnel) by condition, with thematic analysis of the open-ended items. RQ3 (H5) Spearman-correlates the trust composite and `trust_speed_perceived` against measured normalized time and correctness, characterizing divergence. Given three between-subjects cells at $N \approx 7$–10 each, we lead with effect sizes and 95% CIs over null-hypothesis tests and pre-register the analysis.

Table 4. Scoping manipulation check: share of agent turns whose `agent_context_sent` snapshot contained each source, by condition. Cells outside the condition's allowlist must be 0%. [PLACEHOLDER]

Source in snapshot	C_1	C_2	C_3
Chat	—	—	—
Board	0% (expected)	—	—
GitHub	0% (expected)	0% (expected)	—



Fig. 2. Baseline-normalized task time by condition (C_1/C_2/C_3) and task type (comprehension / single-surface / cross-surface). Error bars show 95% CI. The predicted condition × task-type interaction concentrates on cross-surface tasks. [PLACEHOLDER]

Table 5. Per-task correctness (0–3 rubric) by condition and task type, and within-person NASA-TLX delta (agent − baseline) by condition. [PLACEHOLDER]

Measure	C_1	C_2	C_3
Correctness, cross-surface (mean)	—	—	—
Correctness, single-surface (mean)	—	—	—
Correctness, comprehension (mean)	—	—	—
ΔNASA-TLX (agent − baseline)	—	—	—

Table 6. Reliance measures by condition: delegation ratio and the accept/edit/override breakdown of agent suggestions. [PLACEHOLDER]

Measure	C_1	C_2	C_3
Delegation ratio (median)	n/a	—	—
Suggestions accepted (%)	—	—	—
Output edited before use (%)	—	—	—
Action undone / overridden (%)	—	—	—

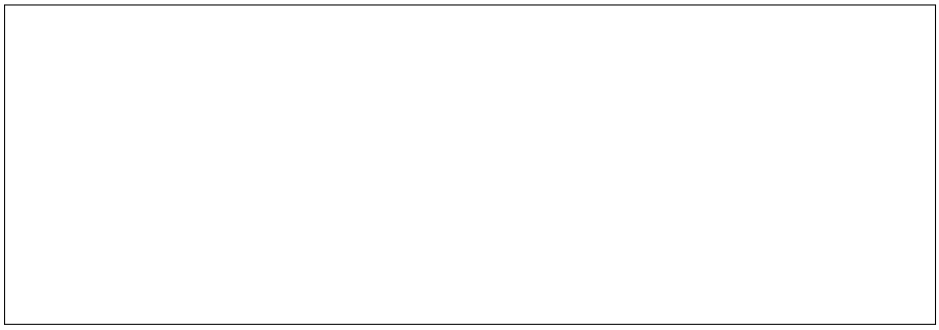


Fig. 3. Perception–behavior gap: perceived speedup / AI-trust score (y) vs. measured baseline-normalized time (x), per participant, colored by condition. Divergence from the diagonal is the H5 finding. [PLACEHOLDER]

Table 7. AI-trust and integration-perception scale means (SD) by condition, with cross-surface telemetry counts for triangulation. [PLACEHOLDER]

Measure	C_1	C_2	C_3
AI trust scale (mean)	—	—	—
`trust_speed_perceived`	—	—	—
Integration perception (mean)	—	—	—
Cross-surface events (median)	—	—	—

5 Results

5.1 Manipulation Check: Context Scoping Held

5.2 Participant Demographics and Data Quality

5.3 RQ1: Context Breadth and Effectiveness

5.4 RQ2: Reliance and Division of Labor

5.5 RQ3: Trust versus Reality

6 Discussion

6.1 Context Breadth as a Design Variable (RQ1)

6.2 Task Type as a Moderator (H2)

6.3 Reliance and Division of Labor (RQ2)

6.4 Trust versus Reality (RQ3 / H5)

6.5 Code-Enforced Context Scoping as a Design Principle

The Context Gate pattern—enforcing context boundaries in server code rather than model prompts—has implications beyond this study. First, it makes the manipulation mechanically verifiable: any reviewer can inspect the `agent_context_sent` audit logs and confirm that a C_1 agent never received board data. Second, it suggests a governance model for production AI agents in collaborative tools, where organizations need auditable controls over what

information agents can observe. Third, it decouples the *scope* of an agent from its *capability*, allowing researchers and practitioners to vary context breadth independently of model capability.

6.6 Limitations

7 Conclusion

We presented Weld, an integrated developer workspace with a built-in study mode, and a hybrid within/between-subjects user study (N = *TBD*) that isolates the effect of agent context breadth on task performance, cognitive load, user reliance, and the calibration of trust against measured effectiveness. The study's primary methodological contribution is the Context Gate: a server-side, code-enforced context-scoping mechanism that makes the agent's information diet both configurable and auditable, enabling a clean between-subjects manipulation without relying on prompt engineering.

Our results [to be reported] suggest [summary of key empirical findings]. Together, these findings have implications for the design of AI agents in collaborative productivity tools, particularly regarding (1) the value of gradually expanding context rather than granting full information access by default, and (2) the importance of audit-logging context assembly decisions as a trust and governance mechanism.

Future work should examine longitudinal effects of integrated AI agents on team coordination patterns, study generalization to professional teams working on real projects, and explore adaptive context scoping—where the agent's information diet expands or contracts based on task characteristics detected at runtime.

Weld and the study infrastructure are open-source and available at https://github.com/weldtool/weld.

Acknowledgments

The authors thank the study participants for their time. [Additional acknowledgements to be added before submission.]

References

[1] John Brooke. 1996. SUS: A "Quick and Dirty" Usability Scale. In *Usability Evaluation in Industry*. Taylor & Francis, 189–194.

[2] Ann Cavoukian. 2009. *Privacy by Design: The 7 Foundational Principles.* Technical Report. Information and Privacy Commissioner of Ontario.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[4] Mary Czerwinski, Eric Horvitz, and Susan Wilhite. 2004. A Diary Study of Task Switching and Interruptions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 175–182. doi:10.1145/985692.985715

[5] Anind K. Dey. 2001. Understanding and Using Context. *Personal and Ubiquitous Computing* 5, 1 (2001), 4–7. doi:10.1007/s007790170019

[6] GitHub. 2022. *Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness.* Technical Report. GitHub, Inc.

[7] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Advances in Psychology*. Vol. 52. Elsevier, 139–183. doi:10.1016/S0166-4115(08)62386-9

[8] Andrew J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE, 344–354. doi:10.1109/ICSE.2007.45

[9] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining Mental Models: A Study of Developer Work Habits. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*. ACM, 492–501. doi:10.1145/1134285.1134355

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.

[11] Melissa Madsen and Shirley Gregor. 2000. Measuring Human-Computer Trust. In *Proceedings of the 11th Australasian Conference on Information Systems*. 6–8.

[12] Gloria Mark, Daniela Gudith, and Ulrich Klocke. 2008. The Cost of Interrupted Work: More Speed and Stress. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 107–110. doi:10.1145/1357054.1357072

[13] Shakked Noy and Whitney Zhang. 2023. Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence. *Science* 381, 6654 (2023), 187–192. doi:10.1126/science.adh2586

[14] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.